

AI Call Agent System

System Design Document

Optimized & Future-Safe Architecture

Version:	1.0
Date:	October 2026
Status:	Final
Organization:	College IT Department
Classification:	Internal Use Only

Contents

1	Design Overview	2
1.1	Purpose	2
1.2	Design Principles	2
1.3	Technology Stack Summary	2
2	High-Level Architecture	2
2.1	Component Diagram	2
2.2	Call Flow — End to End	4
3	Component Design	4
3.1	Telephony Module	4
3.1.1	Plivo Adapter Interface	4
3.1.2	Webhook State Machine	5
3.2	AI Pipeline Module	5
3.2.1	Pipeline Architecture	5
3.2.2	STT Adapter with Fallback	6
3.2.3	LLM Adapter with Fallback	8
3.3	Scheduler Module	8
4	Data Design	9
4.1	Database Schema	9
4.2	Redis Data Structures	11
5	API Design	11
5.1	REST API Endpoints	12
5.1.1	Authentication	12
5.1.2	Campaign Management	12
5.1.3	Call Tasks	12
5.1.4	Plivo Webhooks (internal – not authenticated, IP-whitelisted)	12
5.1.5	Analytics	12
5.2	WebSocket for Real-Time Monitoring	12
6	Deployment Design	13
6.1	Docker Compose Stack	13
6.2	Server Hardware Requirements	14
6.3	Nginx Configuration	14
7	Security Design	15
7.1	Security Controls	15
7.2	Data Privacy	15
8	Scalability & Future-Proofing	16
8.1	Current Capacity	16
8.2	Scaling Strategy	16
8.2.1	Vertical Scaling (when GPU VRAM allows)	16
8.2.2	Horizontal Scaling (future)	17

8.2.3	Model Upgrade Path	17
8.2.4	Provider Redundancy	17
9	Monitoring & Observability	17
9.1	Key Metrics	17
9.2	Grafana Dashboards	17
10	Directory Structure	18
11	Risk & Mitigation	19

Design Overview

Purpose

This document defines the complete system design for the AI Call Agent, covering architecture, component design, data models, API contracts, deployment topology, and scaling strategy. The design is built to be:

- **Deployable today:** runs entirely on a single Ubuntu 22.04 college server with existing hardware
- **Future-safe:** modular interfaces allow swapping AI models, telephony providers, and databases without redesign
- **Cost-efficient:** Rs. 0.76/call using Plivo; cloud AI fallbacks use free tiers
- **Maintainable:** each component is isolated with clear contracts; a junior developer can extend it

Design Principles

1. **Loose Coupling via Adapters:** All external services (telephony, AI models) are accessed through adapter interfaces. Swapping Plivo for Asterisk, or IndicConformer for Whisper, requires changing one adapter file only.
2. **Fail-Fast with Graceful Degradation:** Each AI component (STT, LLM, TTS) has a primary and a cloud fallback. If the primary model fails or is slow, the system falls back automatically.
3. **Event-Driven Call Lifecycle:** The call state machine is driven by webhook events from Plivo. No polling.
4. **Stateless API Layer:** The FastAPI backend holds no call state in memory; all state lives in PostgreSQL. This allows restarting the API without losing active call context.
5. **Separation of Concerns:** Scheduling, telephony, AI pipeline, and staff interface are independent modules.

Technology Stack Summary

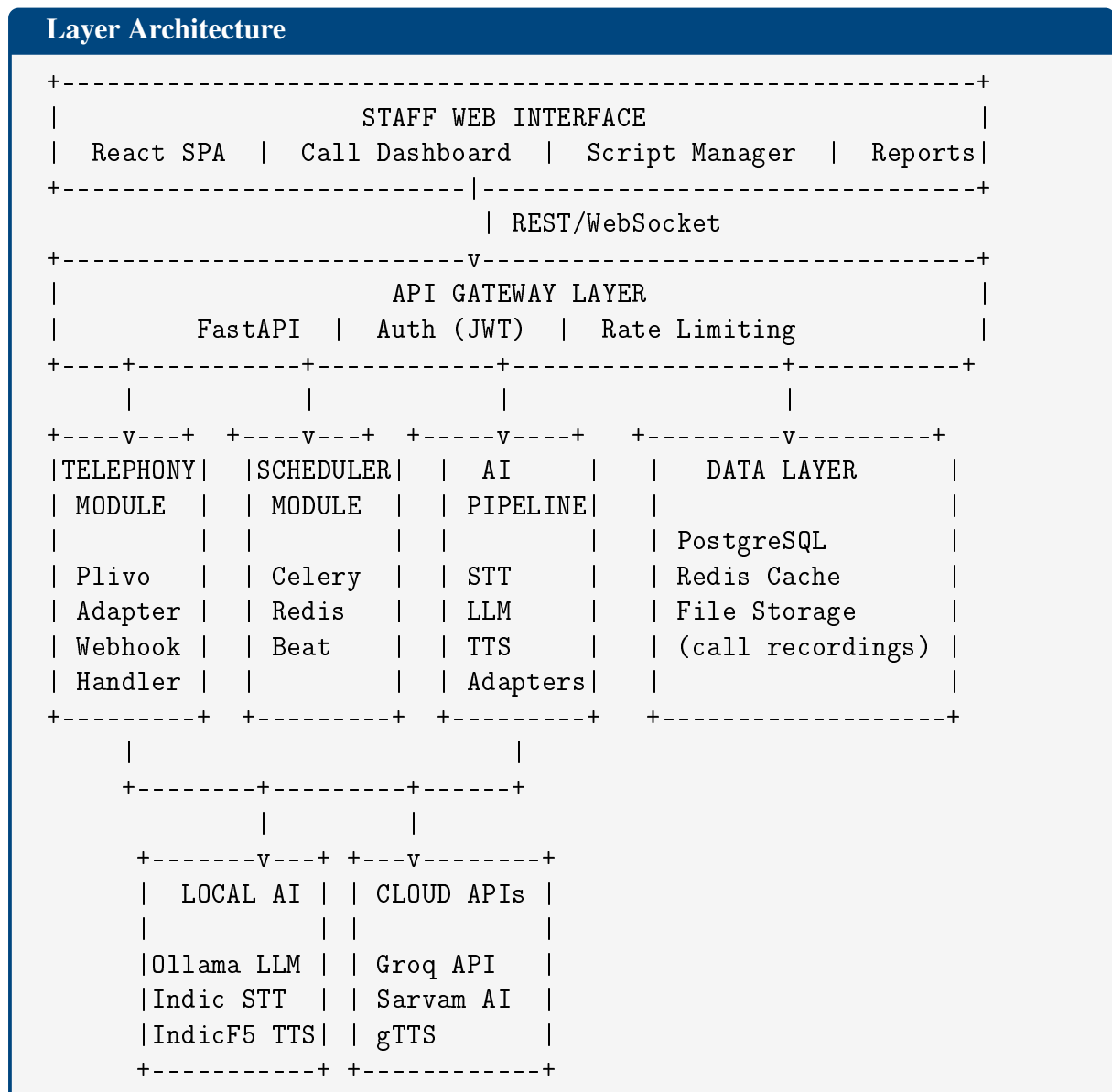
High-Level Architecture

Component Diagram

The system is organized into five layers, each with a clear responsibility boundary:

Layer	Technology	Rationale
API Framework	FastAPI (Python 3.11)	Async support; auto OpenAPI docs
Database	PostgreSQL 15	ACID compliance; JSON support for logs
Task Queue	Celery + Redis	Scheduled dispatch; retry logic
Telephony	Plivo REST API + webhooks	Rs. 0.38/min; reliable India coverage
STT (primary)	IndicConformer 0.60B (local)	Multilingual; offline; fast
STT (fallback)	Sarvam AI API	Free tier 100 min/day
LLM (primary)	Qwen3-14B via Ollama	Runs on 24 GB VRAM; multilingual
LLM (fallback)	Groq API (llama3-70b)	14,400 req/day free; <1s latency
TTS (primary)	IndicF5 0.40B (local)	Natural Hindi/Gujarati voices
TTS (fallback)	gTTS (Google TTS)	Free; always-available
Container	Docker + Docker Compose	Reproducible deployment
Reverse Proxy	Nginx	TLS termination; static files
Monitoring	Prometheus + Grafana	Latency dashboards; alerts

Table 1: Technology Stack



Call Flow — End to End

1. **Scheduler triggers dispatch:** Celery Beat reads the `call_schedules` table at intervals; creates `call_tasks` for due calls.
2. **Telephony module places call:** Celery worker picks the task; Plivo Adapter calls `POST /v1/Call/` on Plivo API with the student number and answer URL.
3. **Call connects – Plivo sends webhook:** On student answer, Plivo POSTs to `/webhook/plivo/answer`. Handler returns an XML response with a `Record` and `GetInput` directive.
4. **Student speaks:** Plivo records audio; sends it to `/webhook/plivo/input` as a URL.
5. **AI Pipeline processes:**
 - STT Adapter downloads audio → runs IndicConformer → returns transcript
 - LLM Adapter sends transcript + context to Qwen3-14B via Ollama → returns reply text
 - TTS Adapter converts reply to audio via IndicF5 → returns audio URL
6. **Response delivered:** Webhook handler returns XML with `Play` directive pointing to the generated audio. Loop continues until call ends.
7. **Call ends:** Plivo POSTs to `/webhook/plivo/hangup`. Handler updates call record with duration, outcome, transcript.
8. **Analytics updated:** Background task writes call summary to `call_logs` and updates student contact history.

Component Design

Telephony Module

Plivo Adapter Interface

The adapter pattern isolates all Plivo-specific logic. If the college switches to Asterisk or Twilio later, only this file changes.

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional

@dataclass
class CallResult:
    call_uuid: str
    status: str # "queued" / "ringing" / "in-progress" / "completed" /
               "failed"
    error: Optional[str] = None

class TelephonyAdapter(ABC):
    """Abstract interface -- all providers implement this."""

    @abstractmethod
```

```

def place_call(self, to_number: str, answer_url: str,
               hangup_url: str, caller_id: str) -> CallResult:
    pass

@abstractmethod
def hangup_call(self, call_uuid: str) -> bool:
    pass

@abstractmethod
def get_call_status(self, call_uuid: str) -> str:
    pass

```

Listing 1: telephony/adapter.py

```

import plivo
from .adapter import TelephonyAdapter, CallResult

class PlivoAdapter(TelephonyAdapter):
    def __init__(self, auth_id: str, auth_token: str):
        self.client = plivo.RestClient(auth_id, auth_token)

    def place_call(self, to_number, answer_url, hangup_url, caller_id)
    -> CallResult:
        try:
            resp = self.client.calls.create(
                from_=caller_id,
                to_=to_number,
                answer_url=answer_url,
                hangup_url=hangup_url,
                answer_method="POST",
                hangup_method="POST",
            )
            return CallResult(call_uuid=resp["request_uuid"], status="
                queued")
        except Exception as e:
            return CallResult(call_uuid="", status="failed", error=str(e)
            ))

```

Listing 2: telephony/plivo_adapter.py

Webhook State Machine

The call lifecycle follows a strict state machine. Invalid transitions are rejected.

AI Pipeline Module

Pipeline Architecture

Each AI stage is a standalone class behind an interface. Stages run sequentially in under 4 seconds total.

```

import asyncio
import time
from .stt import STTAdapter
from .llm import LLMAdapter
from .tts import TTSAdapter

```

Current State	Webhook Event	Next State & Action
SCHEDULED RINGING	call_placed answer	RINGING – set ringing timestamp IN_PROGRESS – start AI pipeline, send first message
RINGING IN_PROGRESS	no_answer (timeout 45s) dtmf_input	NO_ANSWER – mark for retry IN_PROGRESS – process input, generate reply
IN_PROGRESS	speech_input	IN_PROGRESS – run STT+LLM+TTS, send audio
IN_PROGRESS IN_PROGRESS ANY	hangup (student) hangup (timeout 120s) network_error	COMPLETED – save transcript TIMEOUT – save partial transcript FAILED – log error, mark retry

Table 2: Call State Machine Transitions

```

class AIPipeline:
    def __init__(self, stt: STTAdapter, llm: LLMAdapter, tts: TTSAdapter):
        self.stt = stt
        self.llm = llm
        self.tts = tts

    async def process_turn(self, audio_url: str, context: dict) -> str:
        """Returns URL of generated audio response. Target: < 4s total.
        """
        t0 = time.monotonic()

        # Stage 1: STT (target < 1.5s)
        transcript = await self.stt.transcribe(audio_url)

        # Stage 2: LLM (target < 2s)
        reply_text = await self.llm.generate(
            system_prompt=context["system_prompt"],
            history=context["history"],
            user_input=transcript,
        )

        # Stage 3: TTS (target < 0.5s)
        audio_path = await self.tts.synthesize(reply_text, lang=context["lang"])

        elapsed = time.monotonic() - t0
        # Log latency for monitoring
        context["latency_log"].append({"turn": context["turn"], "ms":
            int(elapsed*1000)})

        return audio_path

```

Listing 3: ai_pipeline/pipeline.py

STT Adapter with Fallback

```

import asyncio
from abc import ABC, abstractmethod

```

```

class STTAdapter(ABC):
    @abstractmethod
    async def transcribe(self, audio_url: str) -> str:
        pass

class IndicConformerSTT(STTAdapter):
    def __init__(self, model_path: str):
        import nemo.collections.asr as nemo_asr
        self.model = nemo_asr.models.EncDecCTCModelBPE.restore_from(
            model_path)

    async def transcribe(self, audio_url: str) -> str:
        # Download audio and run inference
        import httpx, tempfile, os
        async with httpx.AsyncClient() as client:
            resp = await client.get(audio_url, timeout=10)
        with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as f:
            f.write(resp.content)
            tmp_path = f.name
        result = self.model.transcribe([tmp_path])
        os.unlink(tmp_path)
        return result[0]

class SarvamSTT(STTAdapter):
    """Cloud fallback - free 100 min/day"""
    def __init__(self, api_key: str):
        self.api_key = api_key

    async def transcribe(self, audio_url: str) -> str:
        import httpx
        async with httpx.AsyncClient() as client:
            resp = await client.post(
                "https://api.sarvam.ai/speech-to-text",
                headers={"api-subscription-key": self.api_key},
                json={"url": audio_url, "model": "saarika:v1", "language_code": "hi-IN"},
                timeout=15,
            )
        return resp.json().get("transcript", "")

class FallbackSTT(STTAdapter):
    """Tries primary; falls back to cloud if primary fails or takes > 2s"""
    def __init__(self, primary: STTAdapter, fallback: STTAdapter):
        self.primary = primary
        self.fallback = fallback

    async def transcribe(self, audio_url: str) -> str:
        try:
            return await asyncio.wait_for(
                self.primary.transcribe(audio_url), timeout=2.0
            )
        except Exception:
            return await self.fallback.transcribe(audio_url)

```

Listing 4: ai_pipeline/stt.py

LLM Adapter with Fallback

```
class OllamaLLM(LLMAdapter):
    def __init__(self, base_url: str = "http://localhost:11434", model:
        str = "qwen3:14b"):
        self.base_url = base_url
        self.model = model

    async def generate(self, system_prompt, history, user_input) -> str:
        import httpx
        messages = [{"role": "system", "content": system_prompt}]
        messages.extend(history[-6:]) # Keep last 3 exchanges
        messages.append({"role": "user", "content": user_input})
        async with httpx.AsyncClient(timeout=5.0) as client:
            resp = await client.post(
                f"{self.base_url}/api/chat",
                json={"model": self.model, "messages": messages, "stream":
                    False},
            )
            return resp.json()["message"]["content"]

class GroqLLM(LLMAdapter):
    """Cloud fallback - 14,400 req/day free"""
    def __init__(self, api_key: str):
        self.api_key = api_key

    async def generate(self, system_prompt, history, user_input) -> str:
        import httpx
        messages = [{"role": "system", "content": system_prompt}]
        messages.extend(history[-6:])
        messages.append({"role": "user", "content": user_input})
        async with httpx.AsyncClient(timeout=10.0) as client:
            resp = await client.post(
                "https://api.groq.com/openai/v1/chat/completions",
                headers={"Authorization": f"Bearer {self.api_key}"},
                json={"model": "llama3-70b-8192", "messages": messages,
                    "max_tokens": 150},
            )
            return resp.json()["choices"][0]["message"]["content"]
```

Listing 5: ai_pipeline/llm.py

Scheduler Module

The scheduler uses Celery Beat for cron-style scheduling. Celery workers handle concurrent call dispatch.

```
from celery import Celery
from celery.schedules import crontab
from datetime import datetime

app = Celery("ai_call_agent", broker="redis://localhost:6379/0")

app.conf.beat_schedule = {
    "poll-due-calls": {
        "task": "scheduler.tasks.dispatch_due_calls",
        "schedule": 30.0, # Every 30 seconds
```

```

    },
}

@app.task(bind=True, max_retries=3, default_retry_delay=300)
def place_call_task(self, call_task_id: int):
    """Celery task to place a single call. Retries up to 3x on failure.
    """
    from database.models import CallTask, db_session
    from telephony.plivo_adapter import PlivoAdapter
    from config import settings

    with db_session() as db:
        task = db.query(CallTask).get(call_task_id)
        if not task or task.status != "pending":
            return

        adapter = PlivoAdapter(settings.PLIVO_AUTH_ID, settings.
                                PLIVO_AUTH_TOKEN)
        result = adapter.place_call(
            to_number=task.student_phone,
            answer_url=f"{settings.BASE_URL}/webhook/plivo/answer?
                        task_id={call_task_id}",
            hangup_url=f"{settings.BASE_URL}/webhook/plivo/hangup?
                        task_id={call_task_id}",
            caller_id=settings.PLIVO_CALLER_ID,
        )

        task.status = "ringing" if result.status == "queued" else "
            failed"
        task.plivo_uuid = result.call_uuid
        task.placed_at = datetime.utcnow()
        db.commit()

        if result.status == "failed":
            raise self.retry(exc=Exception(result.error))

```

Listing 6: scheduler/tasks.py

Data Design

Database Schema

```

-- Students and contact information
CREATE TABLE students (
    id                SERIAL PRIMARY KEY,
    roll_number       VARCHAR(20) UNIQUE NOT NULL,
    name              VARCHAR(100) NOT NULL,
    phone             VARCHAR(15) NOT NULL,
    alt_phone         VARCHAR(15),
    language_pref     VARCHAR(10) DEFAULT 'hi', -- 'hi', 'gu', 'en'
    department        VARCHAR(50),
    semester          INTEGER,
    is_active         BOOLEAN DEFAULT TRUE,
    created_at        TIMESTAMPTZ DEFAULT NOW(),
    updated_at        TIMESTAMPTZ DEFAULT NOW()
)

```

```

);

-- Call scripts / message templates
CREATE TABLE call_scripts (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL, -- fee_reminder, exam_notice,
    etc.
    language VARCHAR(10) NOT NULL,
    system_prompt TEXT NOT NULL,
    opening_message TEXT NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_by INTEGER REFERENCES staff_users(id),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Scheduled call campaigns
CREATE TABLE call_campaigns (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    script_id INTEGER REFERENCES call_scripts(id),
    target_filter JSONB, -- {"department": "CS", "semester": 4}
    scheduled_at TIMESTAMPTZ NOT NULL,
    max_retries INTEGER DEFAULT 2,
    retry_delay_min INTEGER DEFAULT 60,
    status VARCHAR(20) DEFAULT 'pending', -- pending/running/
    completed/cancelled
    created_by INTEGER REFERENCES staff_users(id),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Individual call tasks (one per student per campaign)
CREATE TABLE call_tasks (
    id SERIAL PRIMARY KEY,
    campaign_id INTEGER REFERENCES call_campaigns(id),
    student_id INTEGER REFERENCES students(id),
    student_phone VARCHAR(15) NOT NULL,
    script_id INTEGER REFERENCES call_scripts(id),
    status VARCHAR(20) DEFAULT 'pending',
    -- pending/ringing/in_progress/completed/no_answer/failed/cancelled
    plivo_uuid VARCHAR(50),
    retry_count INTEGER DEFAULT 0,
    scheduled_at TIMESTAMPTZ NOT NULL,
    placed_at TIMESTAMPTZ,
    answered_at TIMESTAMPTZ,
    ended_at TIMESTAMPTZ,
    duration_sec INTEGER,
    outcome VARCHAR(30), -- completed/no_answer/busy/failed/
    voicemail
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Full call transcripts and AI interaction logs
CREATE TABLE call_logs (
    id SERIAL PRIMARY KEY,
    task_id INTEGER REFERENCES call_tasks(id) UNIQUE,
    transcript JSONB NOT NULL DEFAULT '[]',

```

```

-- [{"role": "assistant", "text": "...", "audio_url": "...", "ts": 1234567},
-- [{"role": "student", "text": "...", "audio_url": "...", "ts": 1234568}]]
stt_provider    VARCHAR(20), -- 'indic_conformer' | 'sarvam'
llm_provider    VARCHAR(20), -- 'qwen3' | 'groq'
tts_provider    VARCHAR(20), -- 'indicf5' | 'gtts'
avg_latency_ms  INTEGER,
max_latency_ms  INTEGER,
error_log       JSONB DEFAULT '[]',
created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Staff users
CREATE TABLE staff_users (
    id            SERIAL PRIMARY KEY,
    email         VARCHAR(100) UNIQUE NOT NULL,
    name          VARCHAR(100) NOT NULL,
    role          VARCHAR(20) DEFAULT 'operator', -- 'admin' | 'operator'
    password_hash VARCHAR(255) NOT NULL,
    is_active     BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Performance indexes
CREATE INDEX idx_call_tasks_status ON call_tasks(status);
CREATE INDEX idx_call_tasks_campaign ON call_tasks(campaign_id);
CREATE INDEX idx_call_tasks_scheduled ON call_tasks(scheduled_at);
CREATE INDEX idx_students_phone ON students(phone);
CREATE INDEX idx_call_logs_task ON call_logs(task_id);

```

Listing 7: Complete PostgreSQL Schema

Redis Data Structures

Redis is used for two purposes: Celery message broker and in-flight call session cache.

Key Pattern	Type	Purpose
call:session:{uuid}	Hash	Active call context (history, turn count, start time). TTL: 600s
call:audio:{uuid}:{turn}	String	URL of generated TTS audio. TTL: 300s
celery:{queue}	List	Celery task queues (default, calls, analytics)
rate:plivo:{minute}	Counter	Plivo API calls per minute for rate limiting. TTL: 120s

Table 3: Redis Key Schema

API Design

REST API Endpoints

Authentication

All staff-facing endpoints require a Bearer JWT token obtained via POST /auth/login.

Method	Endpoint	Description
POST	/auth/login	Email + password; returns JWT access token (24h)
POST	/auth/refresh	Refresh token; returns new access token

Campaign Management

Method	Endpoint	Description
GET	/campaigns	List all campaigns with status counts
POST	/campaigns	Create a new campaign
GET	/campaigns/{id}	Get campaign details + task summary
PATCH	/campaigns/{id}/cancel	Cancel a pending/running campaign
GET	/campaigns/{id}/tasks	Paginated list of call tasks

Call Tasks

Method	Endpoint	Description
GET	/calls/{task_id}	Task status + outcome
GET	/calls/{task_id}/transcript	Full conversation transcript
POST	/calls/{task_id}/retry	Manually retry a failed call

Plivo Webhooks (internal – not authenticated, IP-whitelisted)

Analytics

WebSocket for Real-Time Monitoring

Staff dashboard subscribes to live call events via WebSocket:

```
# WebSocket endpoint for real-time call monitoring
@app.websocket("/ws/calls")
async def call_monitor(websocket: WebSocket, token: str = Query(...)):
    user = verify_jwt(token)
    await manager.connect(websocket, user.id)
    try:
        while True:
            await asyncio.sleep(1) # Keep alive
    except WebSocketDisconnect:
        manager.disconnect(websocket)

# Event published to all connected staff when a call state changes
async def broadcast_call_event(event_type: str, task_id: int, data: dict):
    await manager.broadcast({
```

Method	Endpoint	Description
POST	/webhook/plivo/answer	Called when student answers; returns XML
POST	/webhook/plivo/input	Called with speech/DTMF input; returns XML with audio
POST	/webhook/plivo/hangup	Called when call ends; saves transcript
POST	/webhook/plivo/fallback	Called on XML error; graceful hangup

Method	Endpoint	Description
GET	/analytics/daily	Daily call volume, success rate, avg duration
GET	/analytics/latency	STT/LLM/TTS latency percentiles (p50, p95, p99)
GET	/analytics/costs	Estimated costs by day (based on duration * per-min rate)

```

    "type": event_type, # "call_placed" / "call_answered" / "
                        call_ended"
    "task_id": task_id,
    "timestamp": datetime.utcnow().isoformat(),
    **data
})

```

Listing 8: websocket/events.py

Deployment Design

Docker Compose Stack

```

version: "3.9"
services:
  api:
    build: ./backend
    ports: ["8000:8000"]
    depends_on: [postgres, redis]
    environment:
      - PLIVO_AUTH_ID=${PLIVO_AUTH_ID}
      - PLIVO_AUTH_TOKEN=${PLIVO_AUTH_TOKEN}
      - DATABASE_URL=postgresql://user:pass@postgres:5432/ai_calls
      - REDIS_URL=redis://redis:6379/0
      - OLLAMA_BASE_URL=http://ollama:11434
    restart: unless-stopped

  worker:
    build: ./backend
    command: celery -A scheduler.tasks worker -c 4 -Q calls,default
    depends_on: [postgres, redis]
    restart: unless-stopped

  beat:
    build: ./backend
    command: celery -A scheduler.tasks beat --loglevel=info
    depends_on: [redis]
    restart: unless-stopped

  postgres:

```

```
image: postgres:15
volumes: ["pgdata:/var/lib/postgresql/data"]
environment:
  POSTGRES_DB: ai_calls
  POSTGRES_USER: user
  POSTGRES_PASSWORD: ${DB_PASSWORD}
restart: unless-stopped

redis:
  image: redis:7-alpine
  restart: unless-stopped

ollama:
  image: ollama/ollama:latest
  volumes: ["ollama_models:/root/.ollama"]
  deploy:
    resources:
      reservations:
        devices: [{driver: nvidia, count: 1, capabilities: [gpu]}]
  restart: unless-stopped

nginx:
  image: nginx:alpine
  ports: ["80:80", "443:443"]
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf
    - ./certs:/etc/nginx/certs
  depends_on: [api]
  restart: unless-stopped

prometheus:
  image: prom/prometheus:latest
  volumes: ["/prometheus.yml:/etc/prometheus/prometheus.yml"]

grafana:
  image: grafana/grafana:latest
  ports: ["3000:3000"]
  volumes: ["grafana_data:/var/lib/grafana"]

volumes:
  pgdata:
  ollama_models:
  grafana_data:
```

Listing 9: docker-compose.yml (abbreviated)

Server Hardware Requirements

Nginx Configuration

```
server {
    listen 443 ssl http2;
    server_name calls.college.edu;

    ssl_certificate      /etc/nginx/certs/fullchain.pem;
    ssl_certificate_key  /etc/nginx/certs/privkey.pem;
```

Component	Minimum (MVP)	Recommended (Production)
CPU	8-core (Intel i7/Xeon)	16-core
RAM	32 GB	64 GB
GPU VRAM	24 GB (RTX 3090/4090)	24 GB
Storage	500 GB SSD	1 TB NVMe SSD
Network	100 Mbps	1 Gbps
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS

Table 4: Server Requirements

```
# Plivo requires publicly reachable webhook URL
location /webhook/ {
    proxy_pass http://api:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    # Only allow Plivo IP ranges
    allow 54.172.60.0/24;
    allow 52.200.142.0/24;
    deny all;
}

# Staff API
location /api/ {
    proxy_pass http://api:8000;
    proxy_set_header Authorization $http_authorization;
}

# React SPA
location / {
    root /usr/share/nginx/html;
    try_files $uri $uri/ /index.html;
}
}
```

Listing 10: nginx.conf (key sections)

Security Design

Security Controls

Data Privacy

- Student phone numbers stored in database; never logged to files or console
- Plivo does not record calls by default; only TTS audio files are stored (auto-deleted after 30 days)
- All API communication over HTTPS; TLS 1.2+ enforced
- Groq and Sarvam API calls: no PII sent (only transcript text, no student names or numbers)

Threat	Control	Implementation
Unauthorized API access	JWT Authentication	24h tokens; RSA-256 signed; refresh token rotation
Webhook spoofing	IP Whitelist + Signature Verification	Nginx allows only Plivo IP ranges; Plivo sends HMAC-SHA256 signature on each POST
SQL Injection	SQLAlchemy ORM	All queries use parameterized statements; no raw SQL
Credential exposure	Environment Variables	Secrets loaded from .env (not in code/git); Docker secrets in production
Student data exposure	Role-Based Access	Operator: view calls only; Admin: manage campaigns + students
Audio data retention	Retention Policy	Call audio deleted after 30 days; transcripts retained 1 year
Brute force login	Rate Limiting	Slowapi: 5 login attempts per minute per IP

Table 5: Security Controls Matrix

Scalability & Future-Proofing

Current Capacity

With a single server and 4 Celery workers running parallel calls:

Metric	Value
Concurrent active calls	4 (limited by GPU/Celery workers)
Calls per hour	40 (avg 6-min call)
Calls per day (8-hour window)	200-300
College requirement	100-200 calls/day
Headroom	50%+

Table 6: Current Capacity Estimates

Scaling Strategy

Vertical Scaling (when GPU VRAM allows)

Adding a second GPU doubles concurrent AI processing capacity. Increase Celery worker count to match.

Horizontal Scaling (future)

The stateless API design supports horizontal scaling when needed:

1. **Multiple API instances:** Run 2-3 FastAPI containers behind Nginx load balancer
2. **Shared state:** All state in PostgreSQL + Redis (already the design)
3. **Distributed workers:** Celery workers on separate machines consume the same Redis queue
4. **Model serving:** Replace Ollama with vLLM for batched GPU inference – handles 10x more requests per GPU

Model Upgrade Path

Because all AI models are behind adapters, upgrades are drop-in:

Current	Future Upgrade	Migration Effort
Qwen3-14B (Ollama)	Qwen3-32B or LLaMA 3.3-70B	Change model name in config
IndicConformer STT	Whisper Large v3 Turbo	Swap STTAdapter class
IndicF5 TTS	Kokoro TTS or XTTS v2	Swap TTSTAdapter class
Plivo telephony	Asterisk ARI (on-premise)	Replace PlivoAdapter class
PostgreSQL	Same (scalable to millions of rows)	No change needed

Table 7: Model Upgrade Path

Provider Redundancy

If Plivo increases prices or has downtime, the adapter pattern allows switching to any SIP provider in under a day:

- **Exotel:** Rs. 0.50/min; strong India reliability; simple REST API
- **Twilio:** Rs. 4.17/min; higher cost but global reliability
- **Asterisk + BSNL SIM:** Rs. 0/min (only call charges); requires hardware maintenance

Monitoring & Observability

Key Metrics

Grafana Dashboards

1. **Call Operations:** Real-time active calls, hourly volume bar chart, daily success/failure pie
2. **AI Latency:** Per-stage latency time series, p50/p95/p99 panels, fallback rate gauge
3. **Cost Tracker:** Daily Plivo spend (duration * rate), monthly projection, budget threshold line
4. **System Health:** CPU, GPU, RAM, disk I/O, Redis memory

Metric	Target	Alert Threshold
End-to-end latency (p95)	< 4s	> 6s
STT latency (p95)	< 1.5s	> 2.5s
LLM latency (p95)	< 2s	> 4s
Call success rate	> 90%	< 80%
Fallback rate (cloud AI)	< 5%	> 20%
Plivo webhook response time	< 200ms	> 500ms
GPU utilization	60-80%	> 95%

Table 8: Monitoring Metrics and Alert Thresholds

Directory Structure

```

ai-call-agent/
|-- backend/
|   |-- main.py                # FastAPI app entrypoint
|   |-- config.py              # Settings from env vars
|   |-- database/
|       |-- models.py          # SQLAlchemy ORM models
|       |-- migrations/        # Alembic migrations
|   |-- telephony/
|       |-- adapter.py          # Abstract TelephonyAdapter
|       |-- plivo_adapter.py    # Plivo implementation
|       |-- webhook_handler.py  # Plivo webhook routes
|   |-- ai_pipeline/
|       |-- pipeline.py         # AIPipeline orchestrator
|       |-- stt.py              # STT adapters (Indic + Sarvam)
|       |-- llm.py              # LLM adapters (Ollama + Groq)
|       |-- tts.py              # TTS adapters (IndicF5 + gTTS)
|   |-- scheduler/
|       |-- tasks.py            # Celery tasks
|       |-- campaign_runner.py  # Campaign dispatch logic
|   |-- api/
|       |-- auth.py             # JWT auth endpoints
|       |-- campaigns.py        # Campaign CRUD
|       |-- calls.py            # Call task management
|       |-- analytics.py        # Reporting endpoints
|   |-- websocket/
|       |-- manager.py          # WebSocket connection manager
|-- frontend/
|   |-- src/
|       |-- pages/              # Campaign, Dashboard, Transcripts
|       |-- components/        # CallCard, CampaignForm, LatencyChart
|       |-- api/                # Axios API client
|-- docker-compose.yml
|-- nginx.conf
|-- prometheus.yml
|-- .env.example
|-- README.md

```

Listing 11: Project Repository Structure

Risk & Mitigation

Risk	Likelihood	Impact	Mitigation
GPU out of memory (model crash)	Medium	High	OOM guard in pipeline; auto-restart via Docker; fallback to cloud
Plivo webhook unreachable	Low	High	Celery retry up to 3x; fallback: Exotel adapter ready
Student answers but doesn't speak	High	Medium	8s silence timeout; AI sends nudge message; hangup after 2nd silence
Groq free tier exhausted	Low	Medium	Monitor daily usage; upgrade to <i>9/monthpaidtierifneeded</i>
Model output in wrong language	Medium	Medium	Language detection on first turn; restart conversation in detected language
Plivo price increase	Low	Low	Adapter ready for Exotel or Asterisk swap

System Design Document – AI Call Agent v1.0
Approved for development use.